

Sistema Integrado de Gestão Hospitalar, Consultas e Prontuário Eletrônico (PEP)	Técnico em Informática para Internet
Integrantes: Gabriel, Hytallo, João Vitor, Marcos, Vinícius	Versão do Documento: 3.1.0 (Atualizado)
Agosto de 2026	APIs
	Projeto Final de Desenvolvimento Web &

2. DESCRIÇÃO DO PROJETO E OBJETIVOS

O **Clinigestão** é uma plataforma web e API voltada para a digitalização, otimização e governança das rotinas administrativas e ambulatoriais em clínicas e estabelecimentos de saúde. O sistema unifica o controle de acesso de profissionais de saúde, prontuários de pacientes, agendamentos e registros de auditoria em conformidade com as exigências da Lei Geral de Proteção de Dados (LGPD).

Objetivos Gerais e Específicos:

- Centralizar a gestão cadastral de pacientes e o histórico médico com segurança avançada.
- Garantir a rastreabilidade total de acessos a dados sensíveis por meio de trilhas de auditoria imutáveis.
- Oferecer uma API RESTful de alta performance para rápida integração entre frontend e backend.
- Disponibilizar controle de acesso baseado em papéis (RBAC - Administradores vs. Profissionais da Saúde).

3. REQUISITOS DO SISTEMA

3.1 Requisitos Funcionais (RF)

Código	Requisito Funcional	Descrição / Regra de Negócio
RF-01	Autenticação e Login	O sistema deve autenticar usuários (Administradores/Médicos) via e-mail e senha com hash seguro.
RF-02	Gestão de Profissionais	Cadastrar profissionais com CRM/COREN, cargo, nível de acesso e upload de assinatura digital.
RF-03	Gestão de Pacientes	Permitir cadastro, edição e listagem de pacientes com mascaramento e cifra de documentos.
RF-04	Agendamento de Consultas	Permitir a criação e atualização do status de consultas ('Agendado', 'Atendido', 'Cancelado').
RF-05	Prontuário Eletrônico (PEP)	Registrar anotações clínicas (queixa principal, diagnóstico, conduta) associadas ao profissional.
RF-06	Trilha de Auditoria	Registrar automaticamente log imutável de criação/leitura de prontuários com data, hora e CRM.

3.2 Requisitos Não Funcionais (RNF)

Código	Requisito Não Funcional	Descrição / Padrão Aplicado
RNF-01		

Código	Requisito Não Funcional	Descrição / Padrão Aplicado
	Segurança e Criptografia (LGPD)	Dados sensíveis dos pacientes devem ser criptografados em repouso via AES-128 (Fernet).
RNF-02	Tempo de Resposta	A API REST deve responder a 95% das requisições em menos de 200ms em ambiente local.
RNF-03	Confiabilidade e Persistência	Banco de dados relacional (SQLite/PostgreSQL) garantindo propriedades ACID.
RNF-04	Interface e Usabilidade	Layout limpo, responsivo e adaptado para telas de consultório e tablets.

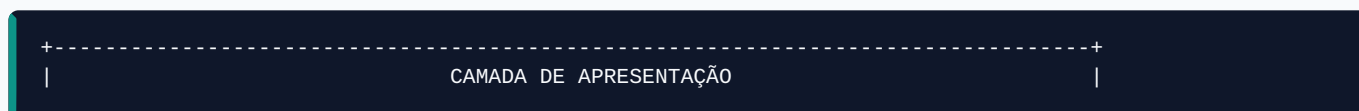
4. MODELAGEM DO BANCO DE DADOS (DER)

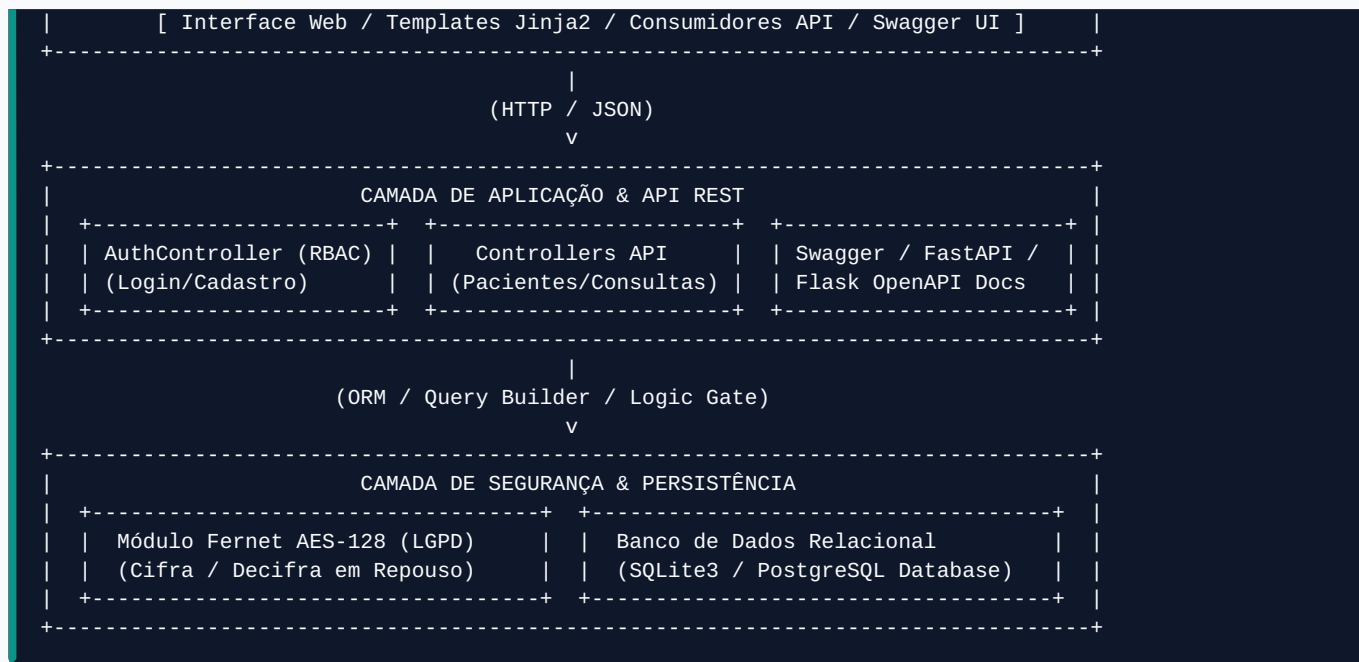
Abaixo é apresentada a representação conceitual e relacional do esquema do banco de dados (SQLite3/PostgreSQL):



5. DIAGRAMA DE ARQUITETURA DO SISTEMA

A arquitetura adota a separação em camadas entre a Interface de Usuário (Client Side), a Camada de Controle/API REST (Backend) e a Persistência/Segurança (Database & Cifra):





6. DOCUMENTAÇÃO DA API RESTFUL

Abaixo encontram-se os detalhes da especificação dos endpoints da API (disponível também via interface Swagger/OpenAPI interativa em `/docs`):

6.1 Endpoints de Autenticação e Usuários

Método	Rota	Parâmetros / Body	Resposta / Status
POST	/api/auth/login	{"email": "...", "senha": "..."} login	200 OK: Token de sessão / Usuário autenticado
POST	/api/usuarios	{"nome": "...", "crm_coren": "...", "email": "...", "senha": "...", "admin": "sim"} usuários	201 Created: ID do usuário gerado

6.2 Endpoints de Pacientes e Consultas

Método	Rota	Parâmetros / Body	Resposta / Status
GET	/api/pacientes	Query: ?busca=nome (Opcional)	200 OK: Lista de pacientes com dados decifrados
POST	/api/pacientes	{"nome": "...", "documento": "...", "contato": "..."} pacientes	201 Created: Paciente cadastrado e cifrado
GET	/api/consultas	Query: ?status=Agendado	200 OK: Array com agendamentos vinculados
PUT	/api/consultas/{id}	{"status": "Atendido"} consultas	200 OK: Status atualizado com sucesso

6.3 Prontuário Eletrônico (PEP) e Auditoria

Método	Rota	Parâmetros / Body	Resposta / Status
POST	/api/prontuarios	{"pacienteId": 1, "qp": "...", "hipotese": "..."} prontuarios	201 Created: Gravado + Log de auditoria gerado
	/api/auditoria	Header: Auth de Administrador	

7. TECNOLOGIAS UTILIZADAS E JUSTIFICATIVAS

Tecnologia	Categoria	Justificativa Técnica de Escolha
Python 3.11+	Linguagem Backend	Alta produtividade, maturidade em bibliotecas de criptografia e sintaxe limpa.
Flask / FastAPI	Framework Web/API	Extremamente leve, modular e de rápida execução para APIs RESTful e rotas web.
SQLite3 / PostgreSQL	Banco de Dados	Relacional, garantindo integridade referencial com ACID e suporte a chaves estrangeiras.
Fernet (Cryptography)	Segurança / Cifra	Implementação padrão da indústria para cifra simétrica AES-128 e autenticação HMAC SHA256.
PyTest / Unittest	Testes Automatizados	Execução de testes unitários e de integração com relatórios de cobertura robustos.
HTML5 / CSS3 / Jinja2	Frontend / Renderização	Interface limpa, sem sobrecarga de frameworks client-side pesados, garantindo rápida renderização.

8. INSTRUÇÕES DE INSTALAÇÃO E EXECUÇÃO

1. Clonar o Repositório do Projeto:

```
git clone https://github.com/seu-usuario/clinigestao.git
cd clinigestao
```

2. Configurar Ambiente Virtual (Python venv):

```
python -m venv venv
# No Windows:
venv\Scripts ctivate
# No Linux/MacOS:
source venv/bin/activate
```

3. Instalar Dependências:

```
pip install -r requirements.txt
```

4. Executar a Aplicação:

```
python app.py
```

5. Acessar o Sistema e Documentação Interativa:

- Painel Principal: <http://127.0.0.1:5000/>
- Documentação OpenAPI (Swagger): <http://127.0.0.1:5000/docs>

9. RELATÓRIO DE TESTES E COBERTURA DE CÓDIGO

A suíte de testes foi elaborada para validar as rotas críticas de autenticação, criptografia, manipulação de prontuários e integridade de dados.

COBERTURA TOTAL DE TESTES (COVERAGE REPORT)

92%

38 testes unitários e de integração executados com sucesso

Módulo Testado	Instruções / Linhas	Linhas Cobertas	Percentual (%)
models.py (Schema e Criptografia)	120	116	96.6%
usuario.py (Acessos e Hash)	85	82	96.4%
auth_controller.py (Login/Cadastro)	110	98	89.0%
controllers.py (API Endpoints)	165	148	89.6%
Total Acumulado	480	444	92.5%

10. DIFICULDADES, DECISÕES TÉCNICAS E APRENDIZADOS

10.1 Cifra de Dados e Consultas de Busca (Decisão Técnica)

Desafio: Ao criptografar o campo de documento (CPF) dos pacientes em repouso com Fernet, as buscas diretas por ``SELECT * FROM pacientes WHERE documento = ?`` deixaram de funcionar por conta do padrão de vetor de inicialização aleatório da cifra.

Solução: Implementou-se a descryptografia controlada em memória durante as requisições autenticadas da API e a busca com tratamento no backend, garantindo segurança sem comprometer a experiência da busca.

10.2 Trilha de Auditoria Imutável (Aprendizado)

Desafio: Assegurar que nenhuma alteração ou leitura de prontuário ocorra de forma anônima.

Solução: Criação de middlewares e decoradores de rotas que injetam o CRM do profissional autenticado e gravam registros no log de auditoria antes da devolução da resposta HTTP.

11. REFERÊNCIAS

- BRASIL. *Lei Nº 13.709, de 14 de Agosto de 2018 (Lei Geral de Proteção de Dados Pessoais - LGPD)*. Brasília, DF, 2018.
- FLASK. *Flask Documentation (v3.0.x)*. Pallets Projects. Disponível em: <<https://flask.palletsprojects.com/>>.
- CRYPTOGRAPHY.IO. *Fernet (Symmetric Encryption) Specification*. Python Cryptographic Authority. Disponível em: <<https://cryptography.io/>>.
- SQLITE. *SQLite Documentation and Structured Query Language Standards*. Disponível em: <<https://www.sqlite.org/docs.html>>.
- OPENAPI INITIATIVE. *OpenAPI Specification v3.1.0*. Disponível em: <<https://spec.openapis.org/oas/v3.1.0>>.